



Department of Mathematics and Computer Science
Interconnected Resource-aware Intelligent Systems

Improving Time Predictability for Time Sensitive Applications in Azure Kubernetes Services

Master's Thesis

Silvia Brighi

Supervisors:

Academic Supervisor: Prof. Dr. Mitra Nasri

Industry Supervisor: Henk van de Crommert, Microsoft

Preparation Phase version

Eindhoven, May 2026

Abstract

Cloud-native platforms such as Azure Kubernetes Service (AKS) are designed to provide scalability, elasticity and efficient resource utilization, while saving costs. However, these platforms inherently lack temporal predictability, which limits their suitability for time sensitive applications, whose correctness depends not only on the logical output but also on the time the output is produced. In public cloud environments, execution-time variability is generated by multiple sources, among which multi-tenancy, virtualization, hypervisor scheduling interference and operating system-level interference, which make it challenging to guarantee bounded execution behaviors.

This thesis investigates how timing predictability can be improved for time-sensitive containerized applications deployed on AKS. More specifically, it explores the integration of real-time scheduling mechanisms, such as KubeDeadline, in combination with infrastructure specific configuration strategies that Azure offers. While KubeDeadline introduces deadline-aware scheduling of pods in Kubernetes through the Linux `SCHED_DEADLINE` policy, its effectiveness has not been studied in public cloud environments where these assumptions are violated.

The methodology used combines experimental and measurement-based approaches to study time-sensitive workloads on public-cloud with KubeDeadline. First, we analyze the impact of public cloud on KubeDeadline scheduling guarantees, to then investigate how KubeDeadline parameters can be derived under non-deterministic conditions. Finally, different Azure IaaS configurations are evaluated to determine how infrastructure-level choices influence timing predictability.

This work contributes both an understanding of the limits of real-time scheduling in public cloud environments and practical guidance for improving predictable execution through combined scheduling and infrastructure design.

Contents

Contents	v
List of Figures	vii
1 Introduction	1
1.1 Problem Statement	1
1.2 Objectives and Research Questions	2
1.3 Contribution	2
1.4 Report Organization	3
2 Background	5
2.1 Real-Time Systems Theory	5
2.1.1 Schedulability Analysis and EDF Scheduling	6
2.1.2 Resource Reservation and Bandwidth Servers	6
2.2 Linux Real-Time Scheduling and Resource Management	7
2.2.1 Control Groups	7
2.2.2 Scheduler and Scheduling Policies	8
2.2.3 SCHED_DEADLINE	9
2.3 Kubernetes Architecture and Scheduling	10
2.3.1 Architecture	11
2.3.2 Pod Scheduling	12
2.3.3 Quality of Service and Resource Enforcement	12
2.3.4 CPU Manager and Static Policy	13
2.4 Azure Infrastructure and AKS	13
2.4.1 Control Plane	13
2.4.2 Node Pools and Virtual Machine Scale Sets	13
2.4.3 Azure Infrastructure Features for Latency Reduction	14
3 Related Works	15
3.1 Cloud Infrastructure Impact on Application Performance	15
3.2 Real-Time Scheduling in Containerized Environments	15
3.3 KubeDeadline	16
4 Methodology	19
4.1 Research Approach	19
4.2 Experimental Environment	20
4.3 Addressing RQ1: KubeDeadline guarantees under cloud conditions	21
4.4 Addressing RQ2: KubeDeadline parameter derivation	22
4.5 Addressing RQ3: Infrastructure configurations for predictability	22
5 Risk Analysis	25
Bibliography	27
Master's Thesis	v

List of Figures

- 2.1 Priority ordering of Linux scheduling classes. The scheduler scans the classes from highest to lowest priority and selects the first non-empty class to choose the next task 9
- 2.2 Components in Kubernetes Cluster 11
- 4.1 Implementation diagram of KubeDeadline in CAPZ 21
- 4.2 Architecture Scenario 0 23
- 4.3 Architecture Scenario 1 23
- 4.4 Architecture Scenario 2 24

Chapter 1

Introduction

Modern cloud-native platforms, such as Azure Kubernetes Service (AKS), have been designed for resource availability, elasticity, and scalability in large distributed environments. Yet an increasingly large class of workloads, namely time-sensitive applications, ranging from online inference pipelines to microservices and distributed control loops, demand something these platforms were never designed to provide: temporal predictability.

At present, public cloud providers face fundamental limitations in providing deterministic execution guarantees, many of which are rooted in the architectural trade-offs of virtualized infrastructure. Nevertheless, there remains meaningful space for improvement within these constraints, and Microsoft is actively investigating how to better serve latency-sensitive workloads of containerized applications on its cloud platform.

As identified by Z. Wan et al. in [36], cloud-native platforms expose workloads to three distinct sources of latency: network, dispatching, and computational latency. While the first two have attracted significant research attention, computational latency and in particular its variance have received comparatively little focus in the context of public cloud deployments.

Time-sensitive applications hosted on AKS suffer from high execution-time variability due to general-purpose operating systems, hypervisor noise [28], Interrupt Requests (IRQ) balancing [36], and multi-tenant interference [3], resulting in unpredictable end-to-end response times.

It follows that AKS does not expose any real-time scheduling primitives that would enable the deployment of time-sensitive containerized applications in a public cloud setting.

Recently, Samimi et al. [30] proposed KubeDeadline, a solution for enabling real-time scheduling in containerized Kubernetes clusters by allowing pods to be scheduled at the operating system level using an EDF-based policy, namely `SCHED_DEADLINE`, available in all mainline Linux kernels. This mechanism provides temporal isolation during task execution while enforcing CPU admission control to guarantee system schedulability.

1.1 Problem Statement

This thesis investigates how to improve timing predictability in AKS, with possible approaches to bring timing predictability in a cloud-native distributed context by introducing KubeDeadline on AKS.

To date, KubeDeadline has been evaluated exclusively in on-premise and localized computing platforms, where interference factors are minimal compared to a public cloud environment such as Azure. The real-time systems research community currently offers many tools and techniques for leveraging and providing predictability to many different environments, but no application or examination has yet been carried out on cloud-native platforms. Moreover, the improvement of determinism at the platform level comes with adjustments to the

infrastructure design.

Therefore, we will examine how infrastructure-level design decisions can contribute to reducing execution-time variance.

This thesis therefore investigates how timing predictability for time-sensitive applications can be improved on AKS through the combination of real-time scheduling mechanisms such as KubeDeadline and deliberate infrastructure configuration.

1.2 Objectives and Research Questions

The overarching goal of this thesis is to improve timing predictability for time-sensitive applications running on AKS, acknowledging and accepting that hard real-time guarantees are not achievable in a public cloud environment.

The following research questions try to address this goal:

- **RQ 1:** Where do KubeDeadline’s scheduling guarantees break down when deployed on AKS?
- **RQ 2:** How to assign parameters of KubeDeadline, such as budget, deadline and period, to time-sensitive tasks in a cloud environment where execution-time characteristics are non-deterministic?
- **RQ 3:** Which Azure IaaS configuration and infrastructure choices most significantly reduce execution-time variability for time-sensitive workloads running under KubeDeadline, and how should these be selected?

1.3 Contribution

This thesis makes a contribution to both academia and industry for two main reasons.

First, the current state-of-the-art for real-time systems has mainly focused on computing in controlled environments with dedicated hardware, either servers or edge nodes, where guarantees on timing and execution can be provided. Yet, little work has been made for public cloud platforms, which represent a new frontier for their highly reliable and cost-effective nature. Therefore, with this thesis, we aim to start a contribution toward this field, which is increasingly moving toward faster and more reliable technologies.

Secondly, at the moment in industry, cloud-native platforms still rely on principles of scalability, elasticity, and throughput, and lack in determinism for time-sensitive applications. Even though these type of applications are becoming increasingly deployed, there is still limited support from public cloud providers.

Therefore, this thesis provides a concrete contribution to industry by providing a design and evaluation for those that require a cloud-native environment for deploying time-sensitive workloads.

The concrete contributions of this thesis will be:

- an analysis of where KubeDeadline guarantees break down in a distributed and dynamic environment like AKS, addressing **RQ1**;
- a principled measurement-based procedure for deriving KubeDeadline parameters (budget, deadline, and period) for time-sensitive tasks in a cloud environment where execution-time characteristics are non-deterministic, addressing **RQ2**;

- a working deployment of KubeDeadline on Azure IaaS via CAPZ, demonstrating the feasibility of RT scheduling in a public cloud environment and providing the experimental foundation for **RQ1**, **RQ2**, and **RQ3**;
- an empirical evaluation of Azure IaaS cluster configurations and their effect on execution-time variability under KubeDeadline, with a ranked comparison of deployment scenarios and concrete infrastructure guidance for practitioners, addressing **RQ3**.

It is worth noting that this thesis considers the following points out of scope:

- providing strong determinism and stringent (hard) timing guarantees. As mentioned earlier, cloud-native platforms are not able to provide guarantees because of their virtualized nature, which inherently leads to unavailability and unpredictability. Nodes depend on virtualized hardware, which is subject to unpredictable node churn and temporary unavailability of the physical processor. This can cause a VM to get deallocated from the CPUs and get assigned to a new processor with a cold cache and restart;
- solutions to reduce network latency or improve fault tolerance through redundancy; similarly, this thesis does not aim to identify or engineer the optimal adaptations required to make KubeDeadline fully compatible with AKS, especially in cases where its mechanisms may not be suitable for a dynamic public-cloud environment;
- extending the research scope to solving specific cloud-infrastructure issues, such as IRQ imbalance, that is the uneven interrupt distribution across CPU cores, as well as storage and network or connectivity problems.

1.4 Report Organization

The remainder of this report is structured as follows: Section 2 presents the background, together with Azure resources available and Kubernetes components; Section 3 surveys related work; Section 4 outlines the methodology applied in order to answer the proposed research questions; Section 5 provides a risk assessment with mitigation strategies.

Chapter 2

Background

This chapter aims at providing the foundational concepts required to understand real-time scheduling in Kubernetes. We begin with the theory of real-time systems and scheduling algorithms, establish how these concepts manifest in the Linux kernel, and finally examine how Kubernetes coordinates scheduling across multiple machines.

2.1 Real-Time Systems Theory

Real-time systems constitute a distinct class of computing systems where correctness depends not only on the logical correctness of the output but also on the time at which the output is produced [7]. In other words, timing is a *correctness criterion* and not only a performance metric. A real-time system must guarantee that all timing requirements are satisfied; failure to produce correct results within specified time bounds represents a system failure, regardless of the logical correctness of the computation.

Real-time systems are characterized by three fundamental properties: *predictability*, *timeliness*, and *robustness*. System behavior must be analyzable and verifiable before runtime, enabling formal verification of timing guarantees through schedulability analysis. Timeliness represents a fundamental correctness requirement and not an optimization criterion. A system that produces correct results too late fails to meet its requirements. Equally important, the system must gracefully degrade under overloads, unpredictable events, or partial failures, while maintaining guarantees for the highest-priority tasks even when resource constraints tighten.

Considering the critical nature of timing constraints, real-time systems are typically classified into three categories based on the consequences of missing a deadline [7].

- **Hard real-time** systems are those in which a missed deadline constitutes a system failure. These systems are found in safety-critical applications such as avionics, automotive systems, and medical devices, where deadline violations can cause catastrophic consequences.
- When a **firm real-time** system misses a deadline, the produced output has zero value and becomes worthless. Missing a deadline may degrade service quality but does not trigger immediate system failure.
- In **soft real-time** systems, missing a deadline reduces the value provided by the task, but the output retains partial usefulness and the system continues to function. Exam-

ples for this category are streaming and interactive services, where occasional deadline misses are tolerable, even though still undesirable.

A real-time task is formally characterized by several fundamental timing parameters [7].

The **period** T_i represents the time interval between successive job releases; for periodic tasks, new instances (jobs) are released at regular intervals of T_i .

The **worst-case execution time (WCET)** C_i is an upper bound on the execution time of a job, representing the maximum time required by the task to execute to completion across all possible inputs and execution conditions, including cache effects and hardware variations. It is worth mentioning that the WCET is a property of a task executing on a specific hardware and software platform, not of the task in isolation. Therefore, the same task may have a different WCET on different processors, operating systems, or runtime environments.

The **relative deadline** D_i is the maximum allowable time interval relative to the job's activation time within which the job must complete, while the **absolute deadline** d_i is the absolute wall-clock time by which the job must complete, computed as the release time plus the relative deadline.

Beyond periodic tasks with fixed inter-arrival times, real-time systems often employ the **sporadic task model**, where tasks are characterized by a minimum inter-arrival time rather than a fixed period: successive jobs arrive with at least T_i time between releases, but exact arrival times are not predetermined. It is the case of event-driven and request-driven workloads, whose activations are triggered by external events.

2.1.1 Schedulability Analysis and EDF Scheduling

Schedulability analysis is an essential step of a real-time system design, as it verifies whether a set of real-time tasks with specified timing parameters and a particular scheduling algorithm executing on a given processor can guarantee all deadlines under all possible arrival scenarios. A critical metric in schedulability analysis is *utilization*, defined as the fraction of processor time allocated to executing real-time tasks. For a single processor, the utilization of a task set is given by [31]:

$$U = \sum_{i=1}^n \frac{C_i}{T_i}$$

Utilization provides a simple yet powerful metric for determining schedulability.

The **Earliest Deadline First (EDF)** scheduling algorithm dispatches jobs by selecting the job with the earliest absolute deadline at each scheduling point [31]. EDF is optimal for single-processor scheduling of periodic and sporadic tasks: if any scheduling algorithm can guarantee that all deadlines are met for a given task set, then EDF will also meet all deadlines [31]. This optimality property assumes preemptive scheduling and no interprocess communication. For periodic task sets on a single processor under EDF, a sufficient and necessary condition for schedulability is $U \leq 1$; a task set is schedulable if and only if the total processor utilization does not exceed 100%, representing the maximum achievable utilization bound for any deterministic scheduling algorithm [31].

2.1.2 Resource Reservation and Bandwidth Servers

In practical systems, real-time tasks often coexist with non-real-time best-effort tasks on the same processor and without explicit isolation mechanisms, best-effort tasks can interfere with real-time tasks and prevent deadlines from being met.

The **Constant Bandwidth Server (CBS)** is a resource reservation mechanism that addresses this by assigning each task a reserved bandwidth defined by a budget Q_i and a period T_i [2]. The task is guaranteed to receive Q_i units of processor time within every period

of length T_i , and is suspended if it exhausts its budget before the period expires. This provides *temporal isolation*, meaning that the timing behavior of one task cannot cause deadline misses in another, regardless of whether tasks are well-behaved or overrun their expected execution times. In Section 2.2.3, CBS parameters in Linux Kernel will be discussed.

CBS comes in two flavors: *soft* and *hard* CBS. In soft CBS, when the budget is exhausted, it is immediately replenished and the server deadline is postponed by T_i . If this happens repeatedly, the deadline keeps being pushed forward, causing *deadline aging* and unbounded delay, since the server remains continuously active as long as it has pending jobs. When very large deadlines accumulate, the server may starve other servers and eventually miss deadlines.

In contrast, hard CBS suspends the server when the budget is exhausted, ensuring a *hard-reservation* behavior. The server only becomes eligible again at its replenishment time, which guarantees a bounded service delay of $\Delta = 2(T - Q)$ and prevents deadline aging by enforcing strict replenishment timing. This formulation corresponds to the Hard Constant Bandwidth Server (H-CBS) described by Biondi et al. [5]. Hard CBS provides higher predictability and is the variant implemented in the Linux kernel's SCHED_DEADLINE scheduler.

2.2 Linux Real-Time Scheduling and Resource Management

The Linux kernel provides several mechanisms that are directly relevant to the scheduling and resource management of real-time tasks in containerized environments. This section covers control groups, which form the foundation of resource isolation in Linux, and the kernel scheduling architecture, with particular focus on the SCHED_DEADLINE policy and the supporting mechanisms relevant to real-time workloads.

2.2.1 Control Groups

Control groups (cgroups) are a Linux kernel mechanism for organizing processes hierarchically and distributing system resources along that hierarchy in a controlled and configurable manner [33]. Each process in the system belongs to exactly one cgroup, and resource controllers attached to a cgroup enforce limits and guarantees on the resources available to the processes within it. Cgroups serve two complementary purposes: *resource limiting*, which prevents a group of processes from consuming more than an allocated share of a resource, and *resource isolation*, which ensures that the resource consumption of one group does not affect the guarantees provided to another.

Hierarchy and Structure Cgroups are organized into a tree hierarchy in which every node represents a cgroup and every process belongs to exactly one node. Resource constraints are inherited top-down, where a child cgroup cannot be granted more of a resource than its parent holds, and enabling a controller on a cgroup causes that controller's policies to apply to all processes in the entire descendant subtree. This hierarchical delegation model allows system administrators to partition resources across multiple levels of organizational structure by, for instance, allocating CPU budgets first by application group and then by individual service within each group.

Two versions of cgroups coexist in the current Linux kernel. Cgroups v1 organizes controllers into independent parallel hierarchies, where each controller maintains its own separate hierarchy [32].

Cgroups v2, instead, consists a unified hierarchy in which all controllers reside together, which provides hierarchical consistency and coordinated resource management the core design principle [33].

Cgroups v2 enforces three key structural invariants to ensure predictability. First, controllers must be explicitly activated on a per-subtree basis via the `cgroup.subtree_control` interface, preventing accidental inheritance of controller policies across subtree boundaries. Second, the no-internal-process rule requires that processes may only reside in leaf nodes of the hierarchy, while intermediate nodes are used only for resource delegation to prevent resource competition between a parent cgroup and its children. Third, resource distribution follows a strict top-down delegation model in which a parent cgroup can only distribute resources it has itself been granted.

Resource Controllers Controllers are kernel modules that attach to the cgroup hierarchy and enforce policies on a specific class of resource. Each controller exposes its configuration through a set of files in the cgroup virtual filesystem, which user-space tools and container runtimes read and write to configure resource limits. When a controller is enabled on a cgroup, its policies apply to all processes in that cgroup and its entire descendant subtree. The Linux kernel provides controllers for CPU time, memory, block I/O, process count, CPU affinity, hugepages, and several device-specific resources [33]. From the perspective of real-time scheduling in containerized environments, two controllers are of primary relevance.

The **CPU controller** regulates how much CPU time a cgroup's processes may consume. It exposes two interfaces: `cpu.weight`, which controls the relative share of CPU time a cgroup receives under CFS when multiple cgroups compete for the same CPU, and `cpu.max`, which enforces an absolute bandwidth limit specified as a quota and a period, that is the maximum CPU time the cgroup may consume within each period interval. A cgroup that exhausts its quota before the period expires is throttled and its processes are suspended until the next period begins, regardless of whether the CPU is otherwise idle.

The **cpuset controller** restricts which physical CPU cores and memory nodes that a cgroup's processes are permitted to use, specified through the `cpuset.cpus` and `cpuset.mems` interfaces respectively. By pinning processes to predefined CPUs, it is possible to eliminate involuntary migration to other cores and the associated cache invalidation and migration overhead. In the context of real-time workloads, CPU pinning strengthens temporal isolation by ensuring that the physical cores reserved for RT tasks are not subject to interference from unrelated processes.

Cgroups and Container Runtimes Container runtimes, such as `containerd`, use cgroups as the primary mechanism for enforcing the resource limits specified in container configurations. Each container is assigned its own cgroup subtree at creation time, and the runtime translates container resource specifications (e.g. CPU requests, CPU limits, memory limits) into the corresponding cgroup controller parameters. In Kubernetes, pod resource requests and limits are mapped onto cgroup parameters by the `kubelet`: a pod's CPU limit is enforced via `cpu.max` on the pod's cgroup, and CPU pinning for isolated workloads is implemented via the `cpuset` controller when the CPU Manager static policy is enabled. The interaction between these cgroup-level enforcement mechanisms and kernel-level scheduling policies such as `SCHED_DEADLINE` is discussed further in Section 2.3.

2.2.2 Scheduler and Scheduling Policies

The Linux kernel scheduler decides which task runs next, on which processor, and when. It implements a class-based architecture in which scheduling classes are prioritized strictly: a higher-priority class always preempts lower-priority ones. Each CPU maintains its own run-queue, subdivided per scheduling class, so CPUs do not compete for tasks through shared data structures. Load imbalances are corrected periodically by a load balancing mechanism that migrates tasks between CPUs.

The kernel defines the following scheduling classes, in descending order of priority [34]:

1. **Stop class:** the highest priority, reserved for internal kernel operations such as CPU hotplug and load balancing. User tasks are never assigned to this class.
2. **Deadline class** (SCHED_DEADLINE): implements EDF scheduling with CBS-based resource reservation. Tasks specify a runtime, period, and deadline; the kernel guarantees the task receives its reserved CPU budget within each period and throttles it if the budget is exceeded.
3. **Real-time class** (SCHED_FIFO, SCHED_RR): implements fixed-priority preemptive scheduling. Tasks at higher priorities always preempt those at lower priorities, but no deadline or bandwidth enforcement is provided.
4. **Fair class** (SCHED_NORMAL, SCHED_BATCH): the Completely Fair Scheduler (CFS), which allocates CPU time proportionally among tasks based on their assigned weights. CFS optimizes for fairness and throughput rather than bounded latency; CPU time allocation is unpredictable under contention, as a task may receive its share split across multiple small intervals.
5. **Idle class:** runs only when no other task is ready, used for background housekeeping.

When the scheduler selects the next task to run, it iterates through the active classes in this strict priority order and selects the first available task from the highest-priority non-empty class. Figure 2.1 shows the hierarchical structure of the scheduling classes in Linux.

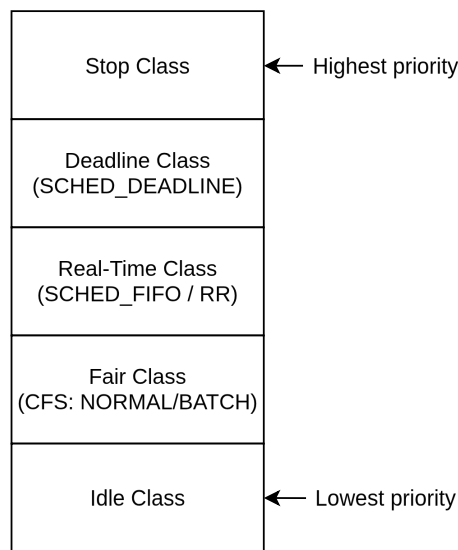


Figure 2.1: Priority ordering of Linux scheduling classes. The scheduler scans the classes from highest to lowest priority and selects the first non-empty class to choose the next task

2.2.3 SCHED_DEADLINE

SCHED_DEADLINE is the Linux kernel's native real-time scheduling class, implementing EDF scheduling with Hard Constant Bandwidth Server (H-CBS) resource reservation [34]. Unlike fixed-priority policies such as SCHED_FIFO and SCHED_RR, which require static priority assignment and become increasingly difficult to reason about as the number of tasks grows, SCHED_DEADLINE allows each task to express its timing requirements directly as scheduling parameters, enabling the kernel to perform optimal dynamic scheduling and admission control without manual priority tuning [20].

Task Parameters When a task is scheduled under `SCHED_DEADLINE`, three parameters must be specified in microseconds [34]:

- **runtime** Q_i : the amount of CPU time the task is guaranteed to receive within each period;
- **period** T_i : the interval within which the runtime budget is made available and replenished;
- **deadline** $D_i \leq T_i$: the time relative to the task's activation by which execution must complete within each period.

The scheduler additionally tracks two dynamic variables per task: the *remaining runtime*, which decrements as the task executes within its current period, and the *scheduling deadline*, which represents the current absolute deadline used for EDF dispatching.

Since `SCHED_DEADLINE` uses the Hard Constant Bandwidth Server (H-CBS) to enforce temporal isolation between tasks [5], each task is treated as a server with a reserved bandwidth $\alpha_i = Q_i/T_i$. As the task executes, its remaining runtime decrements, as explained in Section 2.1.2.

The H-CBS guarantees a bounded service delay of $\Delta = 2(T_i - Q_i)$ for any time interval [5]. This means that the service provided by the server may be shifted by at most Δ time units compared to an ideal virtual processor of speed $\frac{Q_i}{T_i}$. After this initial delay, the server delivers at least Q_i units of execution every T_i time units, ensuring predictable progress as long as the system is not overloaded, which is enforced through admission control.

The key property of CBS is *temporal isolation*: a task that overruns its budget is throttled rather than allowed to consume CPU time beyond its reservation, and a task that completes before exhausting its budget does not influence the scheduling behavior of other tasks. As a result, the timing behavior of one task cannot cause deadline misses in another, provided that the total reserved utilization admitted by the system remains within feasible bounds.

Admission Control Before a task is admitted under `SCHED_DEADLINE`, the kernel performs admission control to verify that accepting the new task will not cause the system to become overloaded [34]. The admission condition requires that the total utilization of all `SCHED_DEADLINE` tasks on a CPU does not exceed a configurable utilization cap:

$$\sum_i \frac{Q_i}{T_i} < \frac{\text{sched_rt_runtime_us}}{\text{sched_rt_period_us}}$$

This cap is configured via global kernel parameters and defaults to 95%, reserving the remaining 5% of CPU time for non-deadline tasks and kernel housekeeping. Admission control is performed both when a task first sets its scheduling parameters via `sched_setattr()` and when a task is migrated between CPUs by the load balancer, ensuring the utilization bound is maintained across core assignments.

2.3 Kubernetes Architecture and Scheduling

Kubernetes is an open-source platform for managing and orchestrating containerized workloads across clusters of machines. It provides mechanisms for automated deployment, scaling, and lifecycle management of containerized applications, and has become the dominant orchestration standard for microservice-based architectures in both on-premise and cloud environments.

Containers are the unit of deployment in Kubernetes. Unlike virtual machines, which virtualize both hardware and operating system, containers share the host operating system kernel while isolating the application's filesystem, network namespace, and process space

through Linux namespaces and cgroups. This makes containers significantly lighter than virtual machines in terms of startup time and resource overhead, while still providing sufficient isolation for multi-tenant workloads.

2.3.1 Architecture

Kubernetes follows a control plane and worker node architecture: the control plane manages the overall state of the cluster and is responsible for scheduling, reconciliation, and cluster-wide coordination, while worker nodes execute the actual workloads and report their state back to the control plane.

The smallest deployable unit in Kubernetes is the **Pod**, that is, one or more containers that are co-located, co-scheduled, and share a common execution context including network namespace, storage volumes, and cgroup hierarchy [17]. Most pods wrap a single container, though tightly coupled containers may be co-located within a single pod when they must share resources directly.

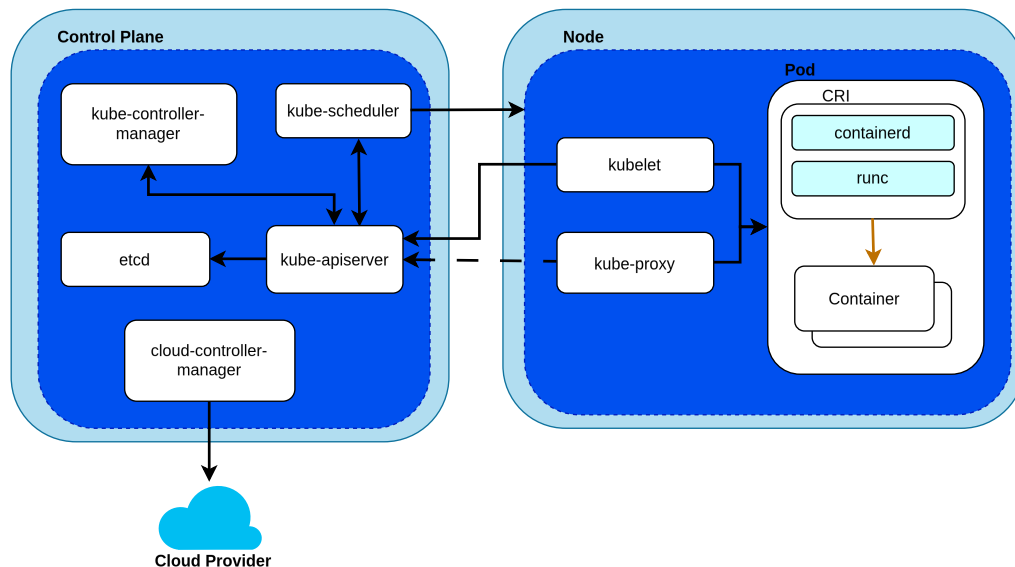


Figure 2.2: Components in Kubernetes Cluster

The control plane consists of the following components, also reported in Figure 2.2:

- **kube-apiserver:** exposes the Kubernetes API and serves as the single entry point for all cluster operations, both from internal components and external clients;
- **kube-scheduler:** monitors unscheduled pods and assigns them to worker nodes based on resource availability and scheduling policies;
- **etcd:** a distributed key-value store that maintains the authoritative state of the cluster configuration and workload status;
- **kube-controller-manager:** runs a set of controllers that continuously reconcile the observed state of the cluster with the desired state, handling conditions such as node failure or pod eviction;
- **cloud-controller-manager:** integrates cluster management with cloud provider-specific infrastructure, such as load balancers and node provisioning.

Each worker node runs the following components:

- **kubelet**: the primary node agent, responsible for ensuring that pods assigned to the node are running and healthy, and for translating pod specifications into container runtime operations including cgroup configuration;
- **kube-proxy**: maintains network rules on the node that implement Kubernetes Service routing, enabling pods to communicate across the cluster;
- **container runtime**: manages the execution and lifecycle of containers on the node operating system. The standard runtime in modern Kubernetes deployments is **containerd**.

2.3.2 Pod Scheduling

Pod scheduling is performed by **kube-scheduler**, which operates in two phases. In the **filtering** phase, the scheduler evaluates all nodes in the cluster against the pod's requirements and produces a list of feasible nodes. In the **scoring** phase, each feasible node is assigned a score based on configurable criteria such as resource utilization balancing, data locality, or custom plugin logic. The node with the highest score gets assigned the pod.

Node affinity and taints are two mechanisms that influence which nodes a pod may be scheduled onto [19]. Affinity is a pod-level property that expresses a preference or hard requirement for scheduling onto nodes with specific labels. Taints are node-level properties that repel pods by default; a pod must declare a matching *toleration* to be eligible for scheduling onto a tainted node. These mechanisms are commonly used to dedicate specific nodes to specific workload types, for instance reserving a set of nodes for latency-sensitive workloads.

The scheduler exposes a plugin framework that allows custom filtering and scoring logic to be injected without replacing the default scheduler. This extension point is the mechanism through which deadline-aware scheduling extensions such as **KubeDeadline** integrate with Kubernetes, and is discussed further in Chapter 3.

2.3.3 Quality of Service and Resource Enforcement

Kubernetes assigns each pod a Quality of Service (QoS) class based on the resource requests and limits specified in the pod specification [18]. This class determines the pod's priority during resource contention and eviction decisions, and directly affects how the kubelet configures cgroups for the pod on the worker node.

Three QoS classes are defined:

1. **BestEffort**: the pod specifies no resource requests or limits. It receives whatever CPU and memory is available and is the first to be evicted under resource pressure.
2. **Burstable**: the pod specifies resource requests but either omits limits or sets limits higher than requests. It is guaranteed at least its requested resources but may consume more when capacity is available. Under eviction pressure, Burstable pods are terminated after BestEffort pods.
3. **Guaranteed**: the pod specifies both requests and limits, with equal values for every container. The kubelet provides this pod with guaranteed access to its declared resources and evicts it only as a last resort.

The QoS class has a direct consequence for cgroup configuration on the node. BestEffort and Burstable pods are placed in shared cgroup subtrees where CPU time is allocated proportionally under CFS. Guaranteed pods, when the CPU Manager static policy is enabled, are eligible for exclusive CPU core allocation.

2.3.4 CPU Manager and Static Policy

The CPU Manager is a kubelet component that controls how CPU resources are allocated to pods at the node level [16]. It operates in two modes: under the **default policy**, all pods share a common CPU pool and are scheduled by the kernel's CFS across all available cores; no exclusive allocation is provided regardless of QoS class. Under the **static policy**, pods in the Guaranteed QoS class that request whole CPU cores are assigned exclusive access to those cores via the cgroup controller. The assigned cores are removed from the shared pool and no other pod may be scheduled onto them.

Static policy CPU allocation has two consequences relevant to real-time workloads. First, it eliminates CPU time sharing between the RT pod and unrelated workloads, removing a major source of scheduling interference. Second, it pins the pod's processes to a fixed set of physical cores, preventing involuntary migration and the associated cache invalidation overhead. Together with `SCHED_DEADLINE` configured at the task level, static CPU Manager policy provides the foundation for CPU-level isolation of real-time workloads in Kubernetes.

Enabling CPU Manager static policy requires the kubelet to be started with the `-cpu-manager-policy=static` flag, which is a node-level configuration. In managed Kubernetes services, access to kubelet flags is typically restricted, limiting the applicability of this mechanism, which is a constraint discussed in the context of AKS in Section 2.4.

2.4 Azure Infrastructure and AKS

Azure Kubernetes Service (AKS) is a managed Kubernetes offering from Microsoft Azure that follows the Platform-as-a-Service (PaaS) model. In this model, Microsoft operates and maintains the Kubernetes control plane on behalf of the customer, while the customer is responsible for defining workloads, configuring node pools, and managing applications. This division of responsibility reduces the operational overhead of running Kubernetes by shifting infrastructure management, control plane availability, and security patching to Azure, allowing customers to focus on application deployment rather than cluster administration.

2.4.1 Control Plane

The AKS control plane consists of the standard Kubernetes control plane components, presented early in this chapter, running on Microsoft-managed infrastructure. These components are provisioned, operated, and updated by Microsoft as part of the managed service. The customer interacts with the cluster exclusively through the Kubernetes API server, using standard tools such as `kubectl` or the Azure CLI, but has no access to the underlying infrastructure on which the control plane runs. Control plane components cannot be modified, replaced, or patched by the customer. Horizontal scaling and high availability of the control plane are handled automatically by Azure.

This managed model has a direct consequence for the scheduling layer: the default `kube-scheduler` instance running in the control plane is operated by Microsoft. While AKS exposes a limited set of supported scheduler plugin configurations [23], the scheduler binary itself and its core configuration cannot be replaced or modified by the customer.

2.4.2 Node Pools and Virtual Machine Scale Sets

Worker nodes in AKS are provisioned as Azure Virtual Machine Scale Sets (VMSS). A node pool is a group of VMs sharing the same configuration (VM SKU, OS disk type, operating system, and node count) and represents the unit of scaling in AKS. A cluster contains at least one system node pool, which runs system-critical pods such as CoreDNS and the metrics server, and optionally one or more user node pools dedicated to application workloads.

The customer selects the VM SKU for each node pool, which determines the number of virtual CPU cores, memory, and available VM-level features such as accelerated networking or premium storage. AKS provisions these VMs using Microsoft-managed OS images, which are maintained and updated by Microsoft. The customer cannot replace the node operating system image with a custom one, nor can they apply kernel patches or install custom kernel modules on AKS-managed nodes.

The kubelet configuration on each node is partially exposed to the customer through a set of supported flags that can be set at node pool creation time [24], whereas parameters outside this supported set cannot be modified. Similarly, the container runtime is managed by Microsoft and cannot be replaced.

2.4.3 Azure Infrastructure Features for Latency Reduction

Azure offers several infrastructure-level features that influence application performance and latency in virtual machine-based deployments, which operate at different layers of the system, ranging from compute placement and networking to storage and hardware capabilities. Understanding these features is necessary context for the infrastructure configuration choices made in the experimental scenarios of this thesis.

Proximity Placement Groups (PPG) provide control over the physical placement of virtual machines within an Azure datacenter. When a set of VMs is associated with a PPG, Azure allocates them on physically close hardware within the same datacenter facility, reducing inter-VM network latency. This is particularly relevant for distributed applications that rely on frequent communication between components deployed across multiple nodes, where physical distance contributes to latency variance [25].

Accelerated networking enables Single Root I/O Virtualization (SR-IOV), which allows network traffic to reach the VM by bypassing the host's virtual network switch. Rather than routing packets through the software-based virtual switch, which adds per-packet processing overhead and CPU utilization, SR-IOV offloads networking tasks to dedicated hardware on the physical Network Interface Card (NIC), which represents a fundamental component of VMs as it allows them to communicate with Azure, the Internet and other on-premises resources. This reduces both network latency and the variance in packet delivery times, and frees CPU cycles that would otherwise be consumed by packet processing on the VM's cores [22].

Managed disks in Azure support multiple performance tiers (e.g. Standard HDD/SSD, Premium SSD/SSD v2, Ultra Disk), each offering different levels of throughput, latency, and input/output operations per second (IOPS). Storage redundancy models further affect performance: Locally Redundant Storage (LRS) replicates data within a single datacenter facility, while Zone-Redundant Storage (ZRS) replicates data across availability zones. For latency-sensitive workloads, LRS is preferable because ZRS write operations must be acknowledged across zone boundaries, introducing additional and non-deterministic write latency.

Host-level disk caching stores frequently accessed disk blocks on storage resources physically close to the virtual machine, typically using a combination of local SSD and memory on the host server [26]. When a cache hit occurs, data is served locally without traversing the Azure storage network to reach the underlying managed disk, resulting in lower and more stable access times. Read-only or read-write caching can be configured per disk depending on the access pattern of the workload.

Chapter 3

Related Works

This chapter presents the state-of-the-art on real-time in containerization and current public cloud infrastructure analysis.

3.1 Cloud Infrastructure Impact on Application Performance

Public cloud platforms represent an active topic for academic research due to the many open problems and possibilities they present. As identified by Wan [36], cloud-native platforms suffer from three types of latency: networking, overhead, and execution latency. Extensive studies have examined performance optimization, with most work focusing on understanding and mitigating latency, interference, and resource variability [4, 21]. For instance, Nikounia et al. [29] analyze specific sources of unpredictability such as hypervisor noise and co-located VM interference, demonstrating slowdowns of up to 16× in over-committed environments. More recent evaluations confirm that network jitter, bandwidth variability, and OS-level noise significantly impact scalability and execution-time stability, even on HPC-oriented instance types [12, 14].

Within this context, research on Azure Kubernetes Service (AKS) has focused primarily on scalability, microservice orchestration, and cloud-native performance engineering. Muniyandi et al. [27] examine load balancing, service discovery, and distributed state management for scalable microservice architectures on AKS, positioning it as a platform optimized for flexible, high-performance cloud applications rather than timing-sensitive workloads. Similarly, engineering work on Kubernetes-oriented microservice placement by Ding et al. [13] concentrates on cost minimization, dynamic resource competition, and dependency-aware scheduling.

However, across the surveyed literature, no work proposes mechanisms to introduce determinism or improve predictability in public cloud platforms or in managed Kubernetes services such as AKS. Existing research consistently treats variability as an unavoidable property of the cloud and focuses instead on statistical characterization, performance tuning, or autoscaling strategies. No existing study attempts to enforce bounded interference, predictable CPU supply, or WCET-oriented resource models in AKS or similar managed Kubernetes offerings.

3.2 Real-Time Scheduling in Containerized Environments

Early real-time systems research focused primarily on embedded and bare-metal platforms, where deterministic hardware environments allowed formal timing guarantees to be established through schedulability analysis [7], [2]. With the advent of virtualization, the community began investigating how real-time techniques could be extended to shared infrastruc-

ture. The advent of virtual machines represented new challenges: multiple VMs sharing a physical host introduced hypervisor scheduling as an additional source of non-determinism that RT theory had not previously addressed [37].

Cucinotta et al. [11] analyzed how to provide predictable timing guarantees to real-time applications running on virtual machines, showing experimentally that when multiple VMs share the same physical host and are scheduled with general-purpose algorithms, deadline misses become frequent and response-time variability increases dramatically. For example, they observed that periodic tasks that met all deadlines on bare metal missed deadlines consistently inside KVM, and response-time distributions exhibited order-of-magnitude fluctuations due to uncontrolled inter-VM interference. Abeni et al. [6] further explored this setting by investigating how to partition real-time tasks across multiple vCPUs and assign CPU reservations to each vCPU in multi-core real-time VMs. Their evaluation showed that global scheduling approaches incur significantly higher allocation overhead, often requiring 30–80% more CPU bandwidth than partitioned scheduling to guarantee the same workload. They also demonstrated that lightweight heuristics, such as First Fit and Best Fit, can partition complex task sets within milliseconds while achieving overheads close to optimal MILP-based solutions, enabling practical online admission control in dynamic cloud/edge environments.

However, VMs carry high deployment overhead, and replicating the entire OS makes them unsuitable for scaling across large numbers of applications with different timing requirements [9]. This motivated the shift toward containerization, which leverages cgroups and namespaces in the Linux kernel to provide isolation at lower cost. Abeni et al. [1] proposed a minimally invasive extension of the `SCHED_DEADLINE` scheduler optimized for containerized applications, while Checconi et al. [8] established the theoretical guarantees of hierarchical multiprocessor reservations, providing the formal foundation for compositional scheduling in containerized environments.

As Kubernetes became the *de-facto* standard for container orchestration, the community extended this work to the Kubernetes layer. Toscano et al. [35] proposed a fixed-priority fully preemptible controller for Kubernetes, enabling mixed-criticality workload separation, though without addressing parameter derivation under cloud-induced execution-time variability. Fiori et al. [15] integrated a hierarchical real-time scheduler based on `SCHED_DEADLINE` into the Kubernetes control plane and worker-node components, enabling containers to receive explicit CPU reservations and representing an important architectural step toward RT containerized workloads.

However, all of these works assume controlled execution environments (e.g. edge deployments, private clouds, or dedicated servers) where hardware is stable and interference is bounded. None address a public cloud environment, where nodes are dynamically managed, CPU cores are virtual and non-persistent, and infrastructure interference cannot be controlled. These are precisely the conditions under which the assumptions of RT scheduling in Kubernetes break down, and which this thesis investigates.

3.3 KubeDeadline

Recently, Samimi et al. [30] introduced KubeDeadline as a modular alternative to RT-Kubernetes [15]. While RT-Kubernetes demonstrated that `SCHED_DEADLINE`-based scheduling was achievable within Kubernetes, it required deep and invasive modifications across multiple core components of the Kubernetes control plane, binding the solution to a single Kubernetes version and making it difficult to maintain.

KubeDeadline addresses this by using Kubernetes' Dynamic Resource Allocation (DRA) framework to implement a real-time resource driver, namely RT-DRA, which performs CBS-based admission control and configures `SCHED_DEADLINE` CPU reservations for containers through standard Kubernetes interfaces, without patching the control plane. Minimal

modifications to `containerd` and `runc` enable the container runtime to apply the reservation parameters at deployment time. The solution relies on the H-CBS kernel patch [1], which extends `SCHED_DEADLINE` to support container-level reservations through a two-level scheduling hierarchy, providing temporal isolation between co-located containers. KubeDeadline is further integrated with KubeRay, enabling distributed machine-learning workloads to benefit from deadline-aware scheduling and CPU isolation.

The evaluation of KubeDeadline was mainly conducted on a cluster of two identical physical servers running a custom real-time Linux kernel with the H-CBS patch, with no hypervisor layer and no virtual CPU abstraction [30]. This controlled bare-metal environment satisfies the assumptions underlying KubeDeadline’s schedulability guarantees: fixed physical CPU cores, a stable and characterizable execution platform, and bounded interference.

These assumptions do not hold in a public cloud environment, where nodes run on virtualized infrastructure managed by a hypervisor, CPU core assignment is non-persistent, the host OS cannot be patched with a custom kernel, and infrastructure interference is outside the operator’s control. The behavior of KubeDeadline under these conditions has not been studied, and it is precisely this gap that the present thesis investigates.

Chapter 4

Methodology

4.1 Research Approach

This thesis follows two methodological parts. First, to address RQ1, we analytically identify the AKS-specific factors that may influence the scheduling guarantees provided by KubeDeadline, and then empirically measure how real cloud conditions affect these factors. Second, to answer RQ2, we derive a procedure for selecting KubeDeadline parameters that ensure applications meet their timing requirements under the observed cloud-induced variability. Secondly, to address RQ3, we define and evaluate two deployment configurations (department-level and enterprise-level) that combine the most relevant infrastructure and cluster-level interventions available on Azure IaaS.

Both parts are grounded in controlled experiments on Azure IaaS, with results reported as execution-time distributions and deadline miss rates rather than formal worst-case bounds, due to the non-deterministic nature of the public cloud environment.

It is worth noting two aspects of this research: the reason measurement-based methods are used and why we target the reduction of variability.

In highly dynamic environments such as the public cloud, deriving a theoretically sound upper bound on the WCET using static-timing analysis (STA) is not currently feasible. STA techniques rely on detailed and stable knowledge of the underlying hardware and execution behavior; however, such assumptions do not hold in modern cloud infrastructures. In particular, current STA approaches struggle to accurately model multicore systems, where contention for shared resources introduces complex and non-deterministic timing effects. This challenge is further amplified in cloud settings, where virtual CPUs may be scheduled on different physical cores, potentially across heterogeneous hardware, and where resource allocation may change during execution. As a result, the factors influencing execution time cannot be fully characterized or bounded analytically. Therefore, a measurement-based approach to WCET estimation is required, in which execution times are empirically observed under representative conditions.

As the number of factors influencing the WCET, such as co-location of workloads, contention of shared resources, and variability in virtual CPU scheduling, can be greater than those analyzed in this thesis, their behavior and interaction may vary over time due to the dynamic nature of the cloud environment. Even with proper tuning of the infrastructure parameters, these effects cannot be completely eliminated. For this reason, we focus on the reduction of the variance in the WCET distribution rather than the determination of the variability of it.

4.2 Experimental Environment

AKS represents the target service on which we want to work. Nevertheless, AKS is a platform-as-a-service (PaaS), and part of its components are Azure-managed, meaning that the customer cannot modify or customize them. While AKS exposes a defined set of configurable parameters, ranging from VM SKU, that is the virtual machine specific configurations, and node pool size to a limited subset of kubelet flags, critical mechanisms required for real-time scheduling, such as custom kernel installation, control plane access, and scheduler replacement, are not available to the customer in a managed service context.

Table 4.1 summarizes the configuration settings and limitations of AKS.

Table 4.1: AKS configuration boundaries

Element	Configurable	Notes
VM SKU	Yes	Determines vCPU count, memory, features
Node pool size and autoscaling	Yes	Min/max node count
OS disk type and size	Yes	Managed disk, ephemeral disk
OS type	Yes	Linux or Windows
Kubelet flags	Partial	Limited supported subset only
CPU Manager policy	Partial	Exposed via node pool flags
Container runtime	No	Containerd, managed by Microsoft
Node OS image	No	Microsoft-managed, cannot be replaced
Kernel version or patches	No	No custom kernel or PRE-EMPT_RT
Control plane components	No	API server, scheduler, etcd inaccessible
CNI plugin	Partial	Choice between Azure CNI variants
Scheduler binary	No	Default kube-scheduler only

Because of the limitations AKS has, the experimental setup is based on IaaS, specifically on the Cluster API Provider Azure (CAPZ) project. CAPZ provides declarative Kubernetes-style APIs for cluster creation, configuration, and management, for both self-managed and managed (AKS) Kubernetes clusters [10].

The self-managed cluster created with CAPZ closely follows the same architecture and configuration of AKS, but instead of being a PaaS, it is an IaaS, which therefore allows us to deploy KubeDeadline on the cluster. In this way, the experimental setup used to answer research question RQ1, RQ2 and RQ3 should be as representative as possible of a real AKS cluster.

The implementation of KubeDeadline on CAPZ follows the architecture presented in Figure 4.1. A custom OS image containing the Linux kernel patched to support H-CBS container reservations is first built and registered in Azure Compute Gallery. The CAPZ AzureMachineTemplate for the RT node pool is configured to use this image, ensuring that RT worker nodes boot with the patched kernel. The KubeDeadline modifications to containerd and runc are installed on the RT worker nodes during node provisioning, enabling the container runtime to apply `SCHED_DEADLINE` reservation parameters at container creation time. The RT-DRA resource driver, consisting of the centralized controller and the kubelet-plugin, is deployed on the cluster control plane and system node pool respectively, where it performs admission control and communicates reservation parameters to the worker node kubelets.

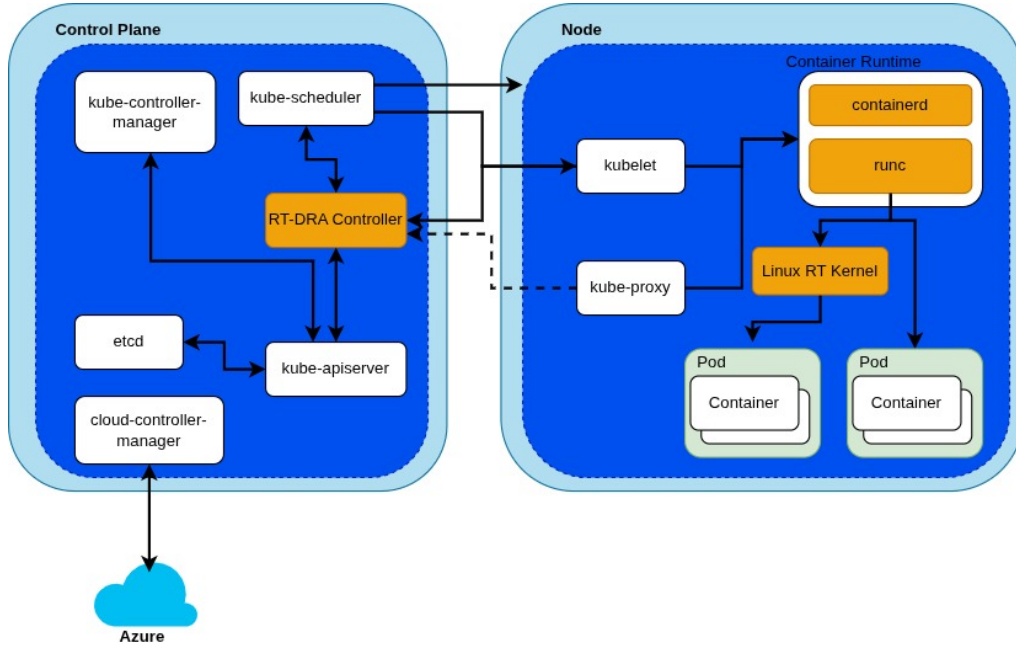


Figure 4.1: Implementation diagram of KubeDeadline in CAPZ

In order to evaluate and quantify the results obtained, the following metrics are presented and will be used across all experiments in both RQ1, RQ2 and RQ3.

- Execution time distribution: represents a primary raw metric that is collected from then the task starts its execution in the CPU till its end.
- Response time distribution: collects the end-to-end time for a task to responde to the request.
- Distribution of deadline miss: this is a correctness metric for assessing both parameter derivation and whether the parameters are correct, as well as for the impact the infrastructure can have on these.

4.3 Addressing RQ1: KubeDeadline guarantees under cloud conditions

As mentioned earlier in this report, KubeDeadline guarantees that real-time containerized workloads meet their required deadlines, assuming that each pod is assigned the following parameters: Budget Q_i , Period P_i , core m_i . However, this statement holds only when the environment is sufficiently isolated and no external factors interfere with the execution of the pod. Moving KubeDeadline to a public cloud platform may jeopardize these guarantees because of multiple additional factors influencing the environment.

During the preparation-phase analysis, we identified several factors that we consider both relevant and measurable. These can be categorized into two types: controllable factors and non-controllable factors. The former consists of those factors that can be varied experimentally through cluster and infrastructure configuration. We can vary them to isolate their individual effect on the execution time variance. These are co-located workload intensity, CPU manager policy and node pool isolation. The latter, instead, consists of those interference factors that cannot be manually changed by the user, because native of public-cloud infrastructures, such as hypervisor interference, CPU migration and cold cache. How

non-controllable factors affect the execution time variation is fundamental for the later step of parameter determination.

These factors are then examined against the execution time distribution, which acts as the dependent variable, in a controlled setup, described previously, using a synthetic real-time workload with periodic tasks.

4.4 Addressing RQ2: KubeDeadline parameter derivation

Once the variability factors have been examined, the focus shifts to the parameter-setting procedure. The outcome of this part is a procedure for estimating a safe upper bound on the execution time from the measurements. The approaches we will investigate are the following:

1. Stress profiling: we run the task under the worst controllable interference scenario to represent the heaviest interference.
2. Tail estimation: from the execution-time distribution, we conservatively estimate the tail, such as P99 or P99.9, with a fixed safety margin. The level of rigor will be evaluated during the experimental part.
3. Inflation factor estimation: by comparing the stress-scenario tail estimate with the baseline median under low interference, we retrieve a ratio. This represents the inflation factor, that is, how much larger the cloud WCET estimate is compared to a stable environment.
4. Parameter estimation: determining appropriate values for Q_i , T_i , and m_i is non-trivial in practice. These parameters are generally set to be smaller than the task deadline, which makes more challenging the derivation from measurement data. In addition, assigning multiple pods to the same node increases resource contention and interference, which complicates more the satisfaction of scheduling conditions. Therefore, rather than providing fixed parameter values, we first investigate the measurement-based approaches introduced above and leave the derivation of a concrete procedure to a later stage, with the objective of meeting timing requirements under realistic cloud conditions.

4.5 Addressing RQ3: Infrastructure configurations for predictability

RQ3 investigates which cluster and infrastructure configuration choices most reduce execution time variability for RT workloads running under KubeDeadline. The focus is entirely on software-level and cluster-level configuration, as hardware-level isolation through dedicated Azure VM SKUs is impractical for the workload profiles and budget constraints representative of the target deployment context. This thesis, therefore, evaluates configurations achievable on standard multi-tenant Azure infrastructure.

The scenarios that will be considered are the following:

- **Scenario 0 — CAPZ baseline.** A self-managed Kubernetes cluster on Azure IaaS with KubeDeadline deployed but no RT-specific infrastructure configuration. Standard compute-optimized VM SKU, default CPU Manager policy, shared node pool where RT and non-RT pods may be co-scheduled, as depicted in Figure 4.2 This represents the minimum viable KubeDeadline deployment and serves as the reference point against which Scenarios 1 and 2 are measured.

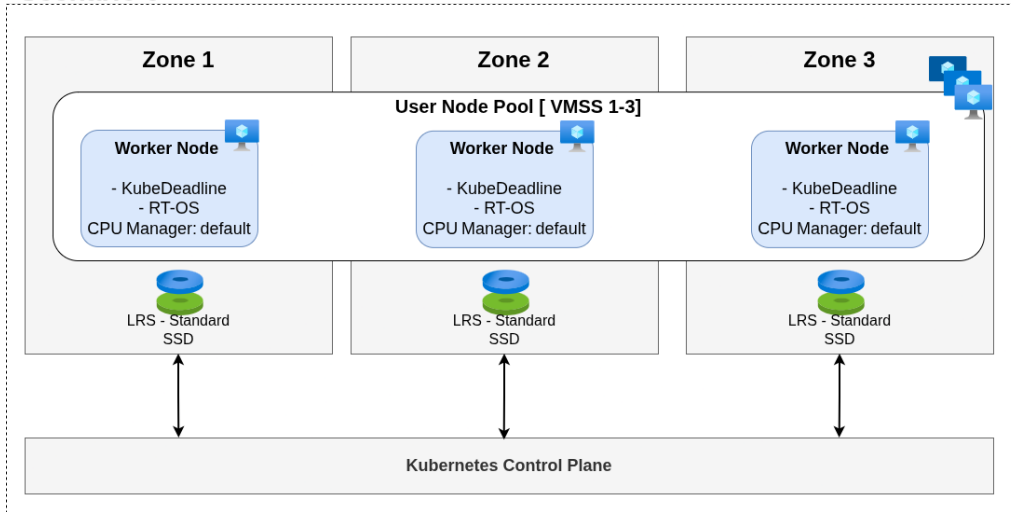
Scenario 0

Figure 4.2: Architecture Scenario 0

- **Scenario 1 — Department cluster.** The same VM SKU and single availability zone as Scenario 0, with software-level RT isolation added: CPU Manager static policy enabled on worker nodes, and a dedicated tainted node pool reserved exclusively for RT workloads through Kubernetes taints and tolerations. A Proximity Placement Group is applied to the RT node pool to request physical co-location of its VMs within the zone. This configuration represents a team-level deployment where RT workloads are isolated at the scheduling and cluster level, reducing CPU contention from co-located workloads, while infrastructure choices remain within standard budget constraints. Figure 4.3 provides the architecture diagram of the scenario.

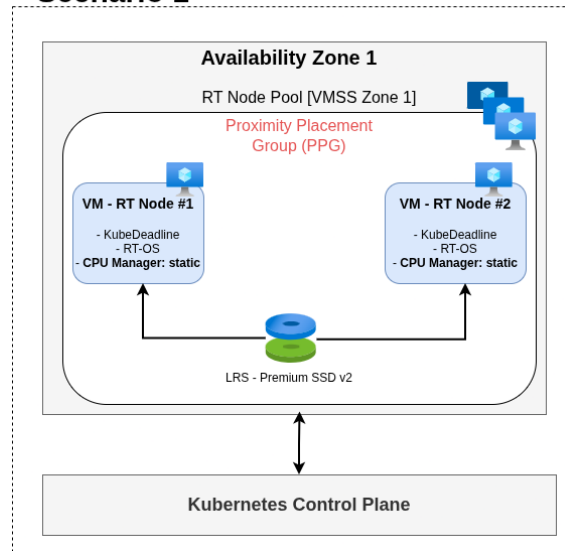
Scenario 1

Figure 4.3: Architecture Scenario 1

- **Scenario 2 — Extended cluster with proximity placement.** All sources of non-determinism that are configurable on standard multi-tenant Azure infrastructure are addressed si-

multaneously. A high-performance compute-optimized VM SKU (Fsv2 or Fxmids family) is selected for stable single-thread CPU performance. CPU Manager static policy and a dedicated tainted RT node pool are retained from Scenario 1. A PPG is applied to the RT node pool. Accelerated networking is enabled on RT nodes, bypassing the virtual network stack via SR-IOV to reduce per-packet processing overhead and network latency variance. Premium SSD v2 with LRS and read caching is used to minimize storage access latency variance for workloads with temporal data locality, such as inference pipelines that repeatedly access the same model weights. In Figure 4.4, we provide the architecture diagram of the scenario.

Scenario 2

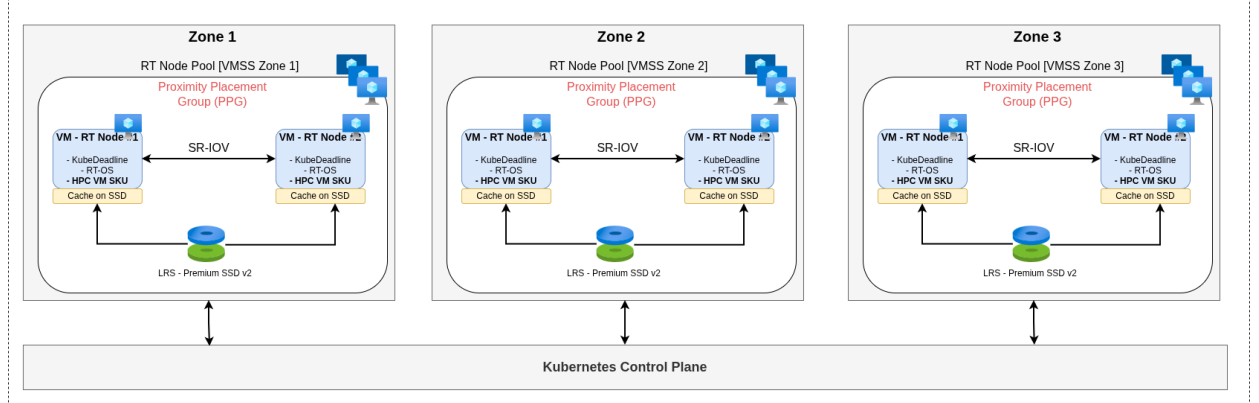


Figure 4.4: Architecture Scenario 2

Chapter 5

Risk Analysis

This work could face challenges during the implementation and therefore, hereby we provide a list of the possible risks and the mitigation strategies to reduce their impact.

KubeDeadline Deployment on Azure IaaS could fail or behave unexpectedly

- KubeDeadline is a research prototype that was deployed and tested on bare-metal servers. It requires a custom RT-Kernel, which can be get by applying the H-CBS kernel patch on a the node OS, modifications to both containerd and runc, and finally, the installation of the RT-DRA of KubeDeadline. These modifications were not designed for a cloud-native platforms, such as Azure IaaS, and, so, getting all these components to work on Azure could require debugging, compatibility works and workarounds that could consume much time of the implementation weeks.
- **Likelihood:** Medium
- **Impact:** Medium/High. It can lead to delays if the problem is particularly challenging and the workaround is not trivial.
- **Mitigation:** This is the highest priority first task of the implementation phase and we will attempt to deploy KubeDeadline on a single Azure IaaS VM before committing to any other implementation work. If workarounds for KubeDeadline implementation are needed, these can be implemented and evaluated as possible alternative to the prototype, specific to cloud-native platforms. If the deployment proves impossible within a reasonable timeframe, the thesis can report why KubeDeadline cannot be deployed on Azure IaaS, which is itself a valid and publishable finding.

Cloud induced variability is too small or too inconsistent to draw meaningful conclusions

- This thesis assumes that execution time variability in AKS is both significantly strong and measurable. If the variance between different conditions is small compared to measurement noise, the experiments could possibly not produce statistically meaningful inflation factors. Conversely, if the variance is too high and extremely unstable, we could report shifts and drifts between the runs, leading to an unreliable analysis.
- **Likelihood:** Medium.
- **Impact:** Medium
- **Mitigation:** Run preliminary experiments on AKS characterization to examine the variance before committing to the full experimental plan. Continuous discussion with the supervisors can help to understand the phenomena and align the experiments.

Reproducibility of the cloud experiment

- On cloud-native platforms, experiments are by nature non-deterministic, which implies that results may vary significantly between identical runs and executions on different days and times.
- **Likelihood:** Medium
- **Impact:** Medium to High
- **Mitigation:** we will collect large samples to reduce the influence of individual outliers and run each experimental condition multiple times on different days and times and possibly report confidence intervals alongside point estimates. If inter-run variance is high, we report this as a characterization of the cloud noise floor rather than a measurement failure.

Bibliography

- [1] Luca Abeni, Alessio Balsini, and Tommaso Cucinotta. Container-based real-time scheduling in the linux kernel. *SIGBED Rev.*, 16(3):33–38, November 2019. 16, 17
- [2] Luca Abeni and Giorgio Buttazzo. Integrating multimedia applications in hard real-time systems. In *Proceedings 19th IEEE Real-Time Systems Symposium (Cat. No. 98CB36279)*, pages 4–13. IEEE, 1998. 6, 15
- [3] Hussain AlJahdali, Abdulaziz Albatli, Peter Garraghan, Paul Townend, Lydia Lau, and Jie Xu. Multi-tenancy in cloud computing. In *2014 IEEE 8th International Symposium on Service Oriented System Engineering*, pages 344–351, 2014. 1
- [4] Malvinder Singh Bali and Shivani Khurana. Effect of latency on network and end user domains in cloud computing. In *2013 International Conference on Green Computing, Communication and Conservation of Energy (ICGCE)*, pages 777–782, 2013. 15
- [5] Alessandro Biondi, Alessandra Melani, and Marko Bertogna. Hard constant bandwidth server: Comprehensive formulation and critical scenarios. In *Proceedings of the 9th IEEE International Symposium on Industrial Embedded Systems (SIES 2014)*, pages 29–37, 2014. 7, 10
- [6] Giorgio Buttazzo, Enrico Bini, and Yifan Wu. Partitioning real-time applications over multicore reservations. *IEEE Transactions on Industrial Informatics*, 7(2):302–315, 2011. 16
- [7] Giorgio C Buttazzo. *Hard real-time computing systems: predictable scheduling algorithms and applications*. Springer, 1997. 5, 6, 15
- [8] Fabio Checconi, Tommaso Cucinotta, Dario Faggioli, and Giuseppe Lipari. Hierarchical multiprocessor cpu reservations for the linux kernel. In *Proceedings of OSPERT 2009*, page 15, 2009. 16
- [9] Marcello Cinque, Raffaele Della Corte, Antonio Eliso, and Antonio Pecchia. RT-CASEs: Container-Based Virtualization for Temporally Separated Mixed-Criticality Task Sets. In Sophie Quinton, editor, *31st Euromicro Conference on Real-Time Systems (ECRTS 2019)*, volume 133 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 5:1–5:22, Dagstuhl, Germany, 2019. Schloss Dagstuhl – Leibniz-Zentrum für Informatik. 16
- [10] Cluster API Provider Azure Maintainers. Cluster api provider azure (capz) documentation. <https://capz.sigs.k8s.io/introduction>. Accessed: 2026-04-30. 20
- [11] Tommaso Cucinotta, Gaetano Anastasi, and Luca Abeni. Respecting temporal constraints in virtualised services. In *2009 33rd Annual IEEE International Computer Software and Applications Conference*, volume 2, pages 73–78, 2009. 16
- [12] Daniele De Sensi, Tiziano De Matteis, Konstantin Taranov, Salvatore Di Girolamo, Tobias Rahn, and Torsten Hoefler. Noise in the clouds: Influence of network performance variability on application scalability. 6(3), December 2022. 15

- [13] Zhijun Ding, Song Wang, and Changjun Jiang. Kubernetes-oriented microservice placement with dynamic resource allocation. *IEEE Transactions on Cloud Computing*, 11(2):1777–1793, 2023. 15
- [14] Jamie Ericson, Masoud Mohammadian, and Fabiana Santana. Analysis of performance variability in public cloud computing. In *2017 IEEE International Conference on Information Reuse and Integration (IRI)*, pages 308–314, 2017. 15
- [15] Stefano Fiori, Luca Abeni, and Tommaso Cucinotta. Rt-kubernetes: containerized real-time cloud computing. In *Proceedings of the 37th ACM/SIGAPP Symposium on Applied Computing, SAC '22*, page 36–39, New York, NY, USA, 2022. Association for Computing Machinery. 16
- [16] Kubernetes Authors. Kubernetes documentation - control cpu management policies on the node. <https://kubernetes.io/docs/tasks/administer-cluster/cpu-management-policies/>. Accessed: 2026-03-24. 13
- [17] Kubernetes Authors. Kubernetes documentation - pods. <https://kubernetes.io/docs/concepts/workloads/pods/>. Accessed: 2026-03-24. 11
- [18] Kubernetes Authors. Kubernetes documentation - quality of service pods. <https://kubernetes.io/docs/tasks/configure-pod-container/quality-service-pod/>. Accessed: 2026-03-24. 12
- [19] Kubernetes Authors. Kubernetes documentation - taints and tolerations. <https://kubernetes.io/docs/concepts/scheduling-eviction/taint-and-toleration/>. Accessed: 2026-03-24. 12
- [20] Juri Lelli, Claudio Scordino, Luca Abeni, and Dario Faggioli. Deadline scheduling in the linux kernel. *Software: Practice and Experience*, 46(6):821–839, 2016. 9
- [21] Shruti Mahambre, Purushottam Kulkarni, Umesh Bellur, Girish Chafle, and Deepak Deshpande. Workload characterization for capacity planning and performance management in iaas cloud. In *2012 IEEE International Conference on Cloud Computing in Emerging Markets (CCEM)*, pages 1–7, 2012. 15
- [22] Microsoft. Azure accelerated networking overview and benefits. <https://learn.microsoft.com/en-us/azure/virtual-network/accelerated-networking-overview>, 2025. Accessed: 2026-04-10. 14
- [23] Microsoft. Azure kubernetes service - scheduling plugins, 2025. Accessed: 2026-04-16. 13
- [24] Microsoft. Core concepts for azure kubernetes service (aks), 2025. Accessed: 2026-04-16. 14
- [25] Microsoft Corporate. Azure documentation - proximity placement groups. <https://learn.microsoft.com/en-us/azure/virtual-machines/co-location>. Accessed: 2026-03-24. 14
- [26] Microsoft Corporate. Azure documentation - virtual machines and disk performance. <https://learn.microsoft.com/en-us/azure/virtual-machines/disks-performance>. Accessed: 2026-03-24. 14
- [27] Venkatesh Muniyandi, Pradeep Kumar Muthukamatchi, and Prashanthi Matam. Scalable microservices architecture using azure kubernetes service (aks). In *2025 International Conference on Computing Technologies Data Communication (ICCTDC)*, pages 1–7, 2025. 15

- [28] Seyed Hossein Nikounia and Siamak Mohammadi. Hypervisor and neighbors' noise: Performance degradation in virtualized environments. *IEEE Transactions on Services Computing*, 11(5):757–767, 2018. 1
- [29] Seyed Hossein Nikounia and Siamak Mohammadi. Hypervisor and neighbors' noise: Performance degradation in virtualized environments. *IEEE Transactions on Services Computing*, 11(5):757–767, 2018. 15
- [30] Nasim Samimi, Luca Abeni, Daniel Casini, Mauro Marinoni, Twan Basten, Mitra Nasri, Marc Geilen, Alessandro Biondi, et al. Enabling containerisation of distributed applications with real-time constraints. *LEIBNIZ INTERNATIONAL PROCEEDINGS IN INFORMATICS*, 335, 2025. 1, 16, 17
- [31] John A Stankovic, Marco Spuri, Krithi Ramamritham, and Giorgio Buttazzo. *Deadline scheduling for real-time systems: EDF and related algorithms*, volume 460. Springer Science & Business Media, 1998. 6
- [32] The Linux Kernel Community. Linux kernel documentation: Control groups v1. <https://www.kernel.org/doc/html/v6.18/admin-guide/cgroup-v1/cgroups.html>. Accessed: 2026-03-08. 7
- [33] The Linux Kernel Community. Linux kernel documentation: Control groups v2. <https://www.kernel.org/doc/html/v6.18/admin-guide/cgroup-v2.html>. Accessed: 2026-03-08. 7, 8
- [34] The Linux Kernel Community. Linux kernel documentation: Deadline task scheduling. <https://www.kernel.org/doc/html/v6.18/scheduler/sched-deadline.html>. Accessed: 2026-03-08. 8, 9, 10
- [35] Stefano Toscano, Luigi De Simone, Marco Barletta, and Marcello Cinque. Preempt-k8s: Pod prioritization for mixed-criticality edge-cloud services. In *2025 28th Euromicro Conference on Digital System Design (DSD)*, pages 206–213, 2025. 16
- [36] Zhitao Wan. Cloud computing infrastructure for latency sensitive applications. In *2010 IEEE 12th International Conference on Communication Technology*, pages 1399–1402, 2010. 1, 15
- [37] Sisu Xi, Justin Wilson, Chenyang Lu, and Christopher Gill. Rt-xen: towards real-time hypervisor scheduling in xen. In *Proceedings of the Ninth ACM International Conference on Embedded Software*, EMSOFT '11, page 39–48, New York, NY, USA, 2011. Association for Computing Machinery. 16